МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра информационных технологий в атомной энергетике (ИТАЭ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ 6

по дисциплине «Автоматизированные системы управления элементами промышленных и административных объектов»

АСУ П по подбору и продаже мультимедиа контента

Студент группы *ИКБО-50-23, Враженко Д.О.* _____
(подпись)

Преподаватель *Щербаков К.В.*

_____
(подпись)

Отчет представлен «___»_____202__г.

Москва 2026 г.

1. Функция, требующая доработки

Выбранная функция: автоматическое дообучение корпоративной нейросети на новых данных.

Обоснование выбора:

- Критичность для интеллектуального ядра системы. Ключевым преимуществом системы является собственная нейросеть, которая должна "понимать" предметную область и контекст. Без механизма дообучения модель со временем устаревает, снижается качество распознавания новых терминов и трендов.

- Наличие необходимых данных. Предусмотрен сбор логов запросов и действий пользователей. Эти данные являются идеальным материалом для дообучения модели, чтобы она лучше соответствовала реальным паттернам поведения пользователей.

- Автоматизация без участия человека. В концепции TO-BE подчеркивается замена человека-менеджера. Однако процесс дообучения модели, хоть и автоматизирован по сбору данных, требует создания программного модуля, который будет запускать этот процесс (возможно, по расписанию или при накоплении критической массы новых данных), контролировать его и разворачивать обновленную модель. Это сложная инженерная задача, выходящая за рамки простого использования готовой библиотеки.

- Соответствие ролям пользователей. Определена роль "Администратор нейросети", в чьи обязанности входит "Обучение и дообучение модели". Разработка данного модуля позволит администратору выполнять эти функции через удобный интерфейс (или автоматический триггер), а не вручную на сервере.

2. Анализ существующих языков программирования и IDE

Для реализации функции дообучения нейросетевой модели (NLP) и интеграции её с основной системой рассмотрим два основных языка и соответствующие IDE.

2.1. Языки программирования

Язык	Преимущества для данной задачи	Недостатки
Python	Абсолютное большинство библиотек для работы с нейросетями (PyTorch, TensorFlow) и NLP (Hugging Face, spaCy) написаны для Python. Простота синтаксиса позволяет быстро прототипировать и экспериментировать с архитектурами. Легко интегрируется с бэкендом на FastAPI. Огромное сообщество.	Более низкая скорость выполнения по сравнению с компилируемыми языками, но для задач обучения на GPU это некритично, т.к. основные вычисления идут на видеокартах. GIL может быть ограничением для многопоточности на CPU, но обучение моделей обычно масштабируется через многопроцессность.
C++	Позволяет более тонко управлять памятью и аппаратными ресурсами. Используется в продакшн-среде для инференса моделей, где важна каждая миллисекунда (например, в LibTorch — C++ API PyTorch).	Разработка и отладка алгоритмов машинного обучения на C++ значительно сложнее и медленнее. Меньший выбор библиотек для NLP по сравнению с Python. Для задачи дообучения, которая выполняется не в реальном времени (фоновый процесс), преимущества в скорости не являются определяющими.

Сравнительный анализ языков программирования показывает, что для реализации задачи автоматического дообучения нейросетевой модели выбор должен основываться на специфике решаемой задачи и требованиях к процессу разработки. Python является безусловным лидером в области машинного обучения благодаря богатой экосистеме специализированных библиотек (PyTorch, Transformers, Datasets), простоте прототипирования и широкому сообществу разработчиков. Эти факторы критически важны для задачи дообучения, которая требует частых экспериментов с архитектурой модели и гиперпараметрами. C++, несмотря на высокую производительность, нецелесообразен для данной задачи, так как основная вычислительная нагрузка ложится на GPU (где скорость выполнения операций определяется библиотеками вроде cuDNN, а не самим языком), а сложность разработки и отладки на C++ значительно замедлит создание и модификацию модуля.

Таким образом, для реализации функции дообучения корпоративной нейросети в составе АСУ «Семантический подборщик» оптимальным и обоснованным выбором является язык Python.

2.2. Интегрированные среды разработки (IDE)

IDE	Преимущества для данной задачи	Недостатки
PyCharm Community	Бесплатная и легковесная версия популярной IDE от JetBrains. Обеспечивает отличную поддержку языка Python: умный автокомплит, навигация по коду, рефакторинг, отладчик. Имеет встроенную поддержку научных библиотек (NumPy, PyTorch) и интеграцию с Jupyter Notebooks, что удобно для экспериментов с моделями. Проста в установке и настройке.	Отсутствуют инструменты для веб-разработки (шаблоны, базы данных) и поддержка баз данных «из коробки», но для задачи дообучения это не критично, так как подключение к БД будет осуществляться через отдельные библиотеки (SQLAlchemy). Нет встроенной поддержки удалённой разработки (Remote Development), однако код можно синхронизировать через Git, а запуск на сервере осуществлять через CI/CD.
Visual Studio Community	Мощная, бесплатная IDE от Microsoft с широкими возможностями для разработки на C++, C#, Python и других языках. Имеет отличный отладчик, профилировщик, поддержку CMake и интеграцию с Git. Для C++ является стандартом де-факто в среде Windows.	Избыточна для чисто Python-разработки, требует много дискового пространства и ресурсов. Ориентирована преимущественно на экосистему Windows, в то время как целевой сервер работает под Linux. Поддержка Python в Visual Studio уступает специализированным IDE по удобству работы с научными библиотеками и Jupyter.

Проведённый анализ показывает, что выбор IDE напрямую зависит от языка реализации и целевой инфраструктуры. Для решения задачи дообучения нейросети на языке Python оптимальным вариантом является PyCharm Community, поскольку эта IDE предоставляет все необходимые инструменты для Python-разработки (интеллектуальное редактирование кода, отладка, поддержка Jupyter и научных библиотек) при нулевой стоимости и простоте использования. Visual Studio Community, несмотря на свою мощь и бесплатность, ориентирована преимущественно на разработку под Windows и языки семейства C++, что делает её избыточной и менее удобной для работы в среде Linux с Python-стеком машинного обучения. Таким образом, для реализации модуля автоматического дообучения АСУ «Семантический подборщик» наиболее рациональным выбором является PyCharm Community.

3. Обоснованный выбор инструментов

На основе проведенного анализа для реализации модуля автоматического дообучения корпоративной нейросети делается следующий выбор.

Язык программирования: Python 3.11+.

Обоснование: Язык является стандартом для задач машинного обучения и обработки естественного языка. Позволит использовать весь необходимый стек библиотек (PyTorch, Transformers) и легко интегрировать модуль с остальными микросервисами системы (RabbitMQ для получения задач, PostgreSQL для чтения логов/метаданных), написанными на том же языке.

Интегрированная среда разработки (IDE): PyCharm Community Edition.

Обоснование: Выбор обусловлен тем, что модуль дообучения будет разрабатываться исключительно на Python. PyCharm Community предоставляет все необходимые инструменты для комфортной разработки на этом языке: интеллектуальное редактирование кода, навигацию, отладку и поддержку научных библиотек (PyTorch, Transformers). Бесплатность IDE важна для учебного проекта и соответствует политике использования открытого ПО. Интеграция с Jupyter Notebooks, доступная в PyCharm, позволит проводить эксперименты с дообучением модели в интерактивном режиме перед встраиванием в основной код. Отсутствие встроенных средств для удалённой разработки компенсируется использованием системы контроля версий Git и пайплайнов непрерывной интеграции (CI/CD), что полностью согласуется с выбранной инфраструктурой (Docker, Kubernetes) и нефункциональными требованиями к автоматизации развёртывания.

4. Выбор и описание библиотек

Библиотека	Назначение	Обоснование / Соответствие требованиям
PyTorch	Основной фреймворк для создания, обучения и дообучения нейросетевых моделей.	Соответствует выбору ПО. Обеспечивает гибкость, необходимую для тонкой настройки (fine-tuning) предобученных трансформеров, и эффективно использует GPU (CUDA).
Transformers (Hugging Face)	Предоставляет тысячи предобученных моделей, классы для их загрузки, токенизации и дообучения.	Позволяет не создавать NLP-модель "с нуля", а взять качественную базовую модель и дообучить её на специфических данных системы (запросы пользователей), что кратно ускоряет разработку и повышает качество.
Datasets (Hugging Face)	Для эффективной загрузки и предобработки больших датасетов (логов запросов) перед подачей в модель. Позволяет работать с данными, которые не помещаются в оперативную память.	Логи запросов могут накапливаться в больших объемах. Библиотека обеспечит эффективную работу с ними, что соответствует требованию масштабируемости.
SQLAlchemy	Библиотека для взаимодействия с базами данных (PostgreSQL). Позволит модулю дообучения читать обезличенные логи запросов и историю взаимодействий из БД для формирования тренировочного датасета.	Необходима для интеграции с Модулем 3 (Аналитика), где хранятся логи.
Pydantic	Для валидации конфигураций и параметров запуска дообучения (количество эпох, размер батча, гиперпараметры), которые может задавать администратор нейросети.	Обеспечит надежность и типобезопасность при настройке процесса дообучения, что важно для стабильности системы.
APScheduler / Celery	Для планирования и запуска задач дообучения. Модуль может работать как фоновый процесс, который получает команду на запуск.	Соответствует архитектуре, где Модуль 3 инициирует процесс. Обеспечивает асинхронность и не блокирует работу других сервисов.
MLflow	Платформа для управления жизненным циклом машинного обучения. Позволит логировать параметры каждой попытки дообучения, метрики качества, а также версионировать полученные модели.	Критически важно для администрирования и воспроизводимости результатов. Администратор нейросети сможет сравнивать разные версии модели и при необходимости откатываться к предыдущей рабочей версии, что повышает надежность системы.

5. Программный код

```
import os
import pandas as pd
import torch
from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments,
)
from transformers import EarlyStoppingCallback
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict
import mlflow
import mlflow.pytorch
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
import logging
import sys

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(sys.stdout)],
)
logger = logging.getLogger(__name__)


class FineTuneConfig(BaseSettings):
    base_model_name: str = "cointegrated/rubert-tiny2"
    data_path: str = "./data/new_queries.csv"
    output_dir: str = "./models/finetuned"
    num_train_epochs: int = Field(3, ge=1, le=10)
    per_device_train_batch_size: int = 8
    per_device_eval_batch_size: int = 8
    learning_rate: float = Field(2e-5, gt=0)
    warmup_steps: int = 100
    logging_steps: int = 10
    eval_steps: int = 50
    save_steps: int = 100
    max_seq_length: int = 128
    early_stopping_patience: int = 3
    mlflow_experiment_name: str = "semantic_picker_finetune"
    mlflow_run_name: str = "auto_finetune"
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore"
    )


config = FineTuneConfig()
logger.info(f"Конфигурация загружена: {config.model_dump()}")

if not os.path.exists(config.data_path):
    os.makedirs(os.path.dirname(config.data_path), exist_ok=True)
    sample_data = """text,label
хочу изучить Python,0
где скачать книгу по Go,2
купить курс по машинному обучению,1
научиться работать с нейросетями,0
посоветуйте видео по Django,2
```

```
хочу приобрести подписку,1
изучение основ программирования,0
стоимость доступа к библиотеке,1
поиск аудиокниг по программированию,2
обучение с нуля,0
"""
    with open(config.data_path, "w", encoding="utf-8") as f:
        f.write(sample_data)
    logger.info(f"Создан пример файла данных: {config.data_path}")


class QueriesDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in
self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)


def load_data(data_path: str, tokenizer, max_length: int):
    logger.info(f"Загрузка данных из {data_path}")
    df = pd.read_csv(data_path)
    if "text" not in df.columns or "label" not in df.columns:
        raise ValueError("CSV должен содержать колонки 'text' и 'label'")
    texts = df["text"].tolist()
    labels = df["label"].tolist()
    encodings = tokenizer(
        texts,
        truncation=True,
        padding=True,
        max_length=max_length,
        return_tensors=None,
    )
    dataset = QueriesDataset(encodings, labels)
    logger.info(f"Загружено {len(dataset)} примеров")
    return dataset


def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    predictions = predictions.argmax(axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
predictions, average="weighted")
    acc = accuracy_score(labels, predictions)
    return {"accuracy": acc, "f1": f1, "precision": precision, "recall":
recall}


def main():
    mlflow.set_experiment(config.mlflow_experiment_name)
    with mlflow.start_run(run_name=config.mlflow_run_name) as run:
        mlflow.log_params(config.model_dump())
        logger.info(f"Загрузка базовой модели: {config.base_model_name}")
        tokenizer = AutoTokenizer.from_pretrained(config.base_model_name)
        model =
AutoModelForSequenceClassification.from_pretrained(config.base_model_name,
```

```
num_labels=10)

        dataset = load_data(config.data_path, tokenizer,
config.max_seq_length)
        train_size = int(0.8 * len(dataset))
        eval_size = len(dataset) - train_size
        train_dataset, eval_dataset = torch.utils.data.random_split(dataset,
[train_size, eval_size])
        logger.info(f"Разделение: train={len(train_dataset)},
eval={len(eval_dataset)}")

        training_args = TrainingArguments(
            output_dir=config.output_dir,
            num_train_epochs=config.num_train_epochs,
            per_device_train_batch_size=config.per_device_train_batch_size,
            per_device_eval_batch_size=config.per_device_eval_batch_size,
            learning_rate=config.learning_rate,
            warmup_steps=config.warmup_steps,
            logging_steps=config.logging_steps,
            eval_strategy="steps",
            eval_steps=config.eval_steps,
            save_strategy="steps",
            save_steps=config.save_steps,
            load_best_model_at_end=True,
            metric_for_best_model="accuracy",
            greater_is_better=True,
            report_to="mlflow",
            run_name=mlflow.active_run().info.run_name,
        )

        trainer = Trainer(
            model=model,
            args=training_args,
            train_dataset=train_dataset,
            eval_dataset=eval_dataset,
            compute_metrics=compute_metrics,

callbacks=[EarlyStoppingCallback(early_stopping_patience=config.early_stoppin
g_patience)],
        )

        logger.info("Начало дообучения...")
        trainer.train()

        model.save_pretrained(config.output_dir)
        tokenizer.save_pretrained(config.output_dir)
        logger.info(f"Модель сохранена в {config.output_dir}")

        mlflow.pytorch.log_model(model, artifact_path="model")
        logger.info("Модель залогирована в MLflow")


if __name__ == "__main__":
    main()
```